ARCHITECTURAL DESIGN

Deliverable Code: D3.2

Prepared by: Alessandro Bacchiega, Francesca D'Errico, Andrea Zorzi (FBK)

Revised by: Claudio Eccher (FBK), Guido Bertolini, Giulia Irene Ghilardi, Chiara Pandolfini (IRFMN)

Version: Final

Date: 29/11/2024

Dissemination level: PUBLIC

Project: eCREAM

enabling Clinical Research in Emergency and Acute care Medicine through automated data extraction

Call: HORIZON-HLTH-2021-TOOL-06

Topic: HORIZON-HLTH-2021-TOOL-06-03

Type of action: HORIZON Research and Innovation Actions

Grant Agreement no. 101057726

Funded by the European Union

Summary

Summary **2**
List of Abbreviations **3**
Executive Summary 4
Introduction 5
Common Architecture Patterns 6
The monolithic architecture 6
The microservice architecture 7
Towards a hybrid approach 7
Architecture Overview 9
Frontend 10
Backend 10
Component Definition and Interaction **11**
External Integration and API Exposure 11
Security and Authentication Mechanisms 11
Platform Modules 12
Software Stack **20**
Frontend 20
Backend 20
Storage 21
User Interface Prototype **22**
User Experience (UX) 22
User Interface (UI) 23
The Prototype 24
Methodology 32
Conclusion 35

List of Abbreviations

Abbreviation	Definition
eCREAM	enabling Clinical Research in Emergency and Acute care Medicine through automated data extraction
ED-EHR	Emergency Department Electronic Health Record
ER	Emergency Room
FHIR	Fast Healthcare Interoperability Resources
OIDC	OpenID Connect
mTLS	Mutual Transport Layer Security
ABAC	Attribute-Based Access Control
JWT	JSON Web Token
JWKS	JSON Web Key Set
BPMN	Business Process Model and Notation
IAM	Identity and Access Management
BFF	Backend for Frontend
EHR	Electronic Health Record
LIS	Laboratory Information System
PACS	Picture Archiving and Communication System
UCD	User-Centered Design
UX	User Experience
UI	User Interface
SPA	Single Page Application
ABCDE	Airway, Breathing, Circulation, Disability, Exposure
ISBAR	Introduction, Situation, Background, Assessment, Recommendation
ECO	Echocardiogram
ECG	Electrocardiogram
ABGA	Arterial Blood Gas Analysis

Executive Summary

This document is the deliverable "D3.2 – Architectural design" of the Horizon Europe project "eCREAM enabling Clinical Research in Emergency and Acute care Medicine through automated data extraction" (hereinafter also referred to as "eCREAM", Grant Agreement no. 101057726).

The deliverable describes the second year of activities carried out in the context of the WP3: Design and Development of the Emergency Department Electronic Health Record (ED-EHR).

The main objective of WP3 is to design and develop the Emergency Department Electronic Health Record (ED-EHR) and deploy it in some EDs in Greece (specifically, Crete) and Italy. The WP3 will provide a working ED-EHR, fully integrated into the ED workflow and able to interoperate with clinical and administrative information systems, which will be used and evaluated in real clinical settings in the last quarter of the project.

In the second year of the project, WP3 activities are organized into three concurrent tasks: T3.2, T3.3, and T3.4. Task 3.2, focused on domain analysis, ended in August 2024 and involved examining the domain of emergency medicine, including actors, roles, workflows, functional and non-functional requirements and data models of the Emergency Room (ER). The main content of this deliverable will be based on the outcomes of task 3.3, which covers the conceptual design of the ED-EHR architecture. Task 3.4, which involves designing and developing the database model, business logic components, and user interface, is ongoing and will continue for two years. The work related to the design of the user interface, though, is included in this deliverable.

The deliverable is structured as follows: it begins with an introduction emphasizing the significance of architectural design for the EHR system in emergency care. The following section discusses common architectural patterns, compares different architectural approaches, and explains the motivations behind the model adopted for the eCREAM-EHR. The Architecture Overview section provides an overview of the system, detailing the roles of the frontend and backend layers. The Component Definition and Interaction section presents a comprehensive overview of the system's components and their interactions. The Software Stack section outlines the technologies selected for developing the EHR application. The User Interface Prototype section outlines the design criteria for the user experience and interface, and illustrates the pages of the mock-up prototype. The Conclusions section outlines the future work and evolution of WP3.

Introduction

The architectural design of an electronic health record system for the emergency department requires careful consideration to meet the complex needs of a fast-paced healthcare environment, as already outlined in Deliverable 3.1.

The process of understanding and implementing the needs of healthcare professionals for the EHR has been dynamic and ongoing. A series of well-structured and collaborative meetings have been held, bringing together professionals from various healthcare sectors to engage in meaningful discussions. Each session has provided an opportunity for these dedicated individuals to voice their unique perspectives, share insights from their daily practice, and outline the specific challenges they face in using health information systems.

During these interactions, healthcare professionals have explored their immediate needs and long-term goals. This has involved considering how they would like an ideal EHR to support their workflows, patient interactions and the overall efficiency of care delivery. From these conversations, patterns have emerged that highlight both universal requirements and specialized needs tied to different medical roles.

The software solution proposed by the development team of FBK will consist of a web application designed for different users, namely physicians and nurses, throughout the various stages of emergency care (e.g., at triage, during the clinical examination or nurse treatment, at the patient discharge, etc.).

The architecture on which the software is going to be implemented will be conceptually divided into the two canonical layers, frontend and backend, plus the data repositories used to store all the information. The frontend, represented by the web application, will provide an intuitive and responsive user interface for healthcare professionals. Users will be provided with secure credentials that enable them to authenticate and operate according to their account attributes in the system. On the other hand, the backend will consist of multiple modules, each responsible for a specific business goal, such as authentication, authorization, and clinical data management.

A separation of concerns-based approach ensures modularity, scalability, and robustness, allowing the system to handle the diverse and dynamic requirements of an emergency room effectively. Moreover, it encompasses the flexibility required by having to deploy the EHR in emergency care departments belonging to different healthcare systems, potentially located in different European countries.

Common Architecture Patterns

In order to determine the most appropriate architecture for the new EHR software, a preliminary analysis has been conducted, investigating the possible choices of architectural patterns that are nowadays widely adopted by enterprises and IT companies.

Legacy architectures present significant challenges and drawbacks, including increased time-to-market due to technical debt and cumbersome maintenance, making releases slow and resource-intensive. Additionally, they pose security and compliance risks, as outdated technology stacks accumulate vulnerabilities, potentially leading to security breaches and compliance issues that can jeopardize business relationships.

To overcome these limitations and modernize the platforms, enterprises have moved to choose loosely coupled and pluggable architectures, as these approaches enable rapid adaptation to change business requirements and facilitate seamless digital transformation. These architectures provide flexibility, allowing systems to evolve efficiently while minimizing disruption to business operations.

While considering the possible modern alternatives, we focused on two main architectural choices: *monolithic* architecture and *microservices* architecture. Microservices architecture consists of independent applications functioning as services, each designed to fulfill a specific business objective, with the ability to be deployed autonomously. In contrast, monolithic architecture combines multiple functionalities into a single deployable unit. While monolithic architecture can simplify initial development and deployment, microservices provide a higher degree of flexibility, allowing for smaller services that can be quickly deployed without affecting other parts of the system.

Both approaches, however, have their strengths and limitations and the final decision should be pondered depending on the specific needs and goals of the platform. The following paragraphs are devoted to giving an overview of the two aforementioned architectural patterns, highlighting their advantages and disadvantages.

The monolithic architecture

In the monolithic approach, the application is structured as a single deployable component unit. The application contains all the possible modules that implement a business functionality. Generally, in a monolithic architecture, the database is unique and shared by all components. Since all modules are packed into a single executable unit, the operations between them are local.

The monolith presents several advantages: the interaction between components is usually simple and therefore easy to troubleshoot, the communications are more efficient because they take place in the same container, and components have no runtime coupling.

On the other hand, this architecture has potential drawbacks: the more it grows in size and complexity, the more its maintenance can become burdensome, hindering the team's

autonomy and requiring strong coordination between developers, since they have to work on the same code base. Moreover, it forces the project to have a single software stack, which limits the flexibility of integrating with other systems. Lastly, in a monolithic application there is no possibility to differentiate the components by their peculiarity (e.g., resource consumption, network bandwidth, storage capacity, etc.), which can limit optimization efforts and reduce the scalability as the system grows.

The microservice architecture

Conversely, in the microservice approach, the application is structured as a multitude of modules, each one designed to implement a specific business functionality. The building blocks of this kind of architecture are called microservices, and they represent a set of components that are independently deployable and loosely coupled. Each microservice has its own dedicated database. Operations can be either local, when handled by a single microservice, or distributed, requiring coordination across multiple microservices.

The advantages of this type of architecture include a clear separation of the business logic between the different subdomains, fostering the autonomy of the developer teams. Furthermore, using small, isolated microservices improves the performance of the deployment pipeline, as components can be built, tested and deployed independently. Additionally, it enables the adoption of different software stacks tailored to the specific needs of each microservice and simplifies the segregation based on their characteristics.

As in monolithic architecture, the microservice approach also presents some pitfalls. Among the most critical aspects, the chain of involved microservices to complete a distributed operation can be complex, hard to troubleshoot, and time-consuming. Moreover, this architecture demands a robust approach to transaction management, ensuring that each microservice knows how to handle errors during distributed operations. Additionally, operations involving multiple microservices are susceptible to failure if any of them becomes unavailable.

Towards a hybrid approach

Choosing the architecture for a long-term project like an EHR is a critical and complex decision. Furthermore, adopting one architectural pattern over the others has significant implications throughout the development process of the software.

The choice of a microservice architecture in an EHR system allows for independent deployment and scaling of individual services (e.g., patient records, billing, and lab results). This flexibility can significantly reduce downtime during updates and improve scalability as the system grows. However, it also introduces complexities, such as managing inter-service communication, handling data consistency, and the need for a robust DevOps pipeline to monitor and manage numerous services.

In contrast, opting for a monolithic architecture might simplify development initially, with a single codebase and centralized data handling. However, as the system scales, maintaining, updating, and scaling portions of the software without affecting the entire system can become more challenging. Changes in one module may require the redeployment of the entire system, leading to longer downtimes and reduced agility.

For these technical reasons, given the level of knowledge acquired during the domain analysis task and considering that we are still in the early stages of development (the second of five years), we decided to adopt a hybrid approach that combines both architectural patterns. Specifically, we will begin developing the EHR application using a monolithic architecture, while designing each functionality as separate modules. This allows us to eventually transition these modules into loosely coupled, highly cohesive components, aligning with the principles of a microservice architecture.

Architecture Overview

This section presents a high-level overview of the system architecture. The platform is structured into two distinct layers, *frontend* and *backend* respectively, that continuously collaborate to ensure the full functionality of the EHR application.

The frontend is the part of the software that users interact with directly. It includes the user interface (UI) and handles the presentation of data, capturing user input, and sending it to the backend for processing. Conversely, the backend layer is the part of the software that handles data processing, business logic, and communication with the database or external services. It operates behind the scenes, receiving requests from the frontend, processing them, and returning the necessary data or results.

Figure 1 provides a schematic representation of the EHR software architecture, highlighting the frontend and backend layers and how their modules are grouped according to their type and role within the system.

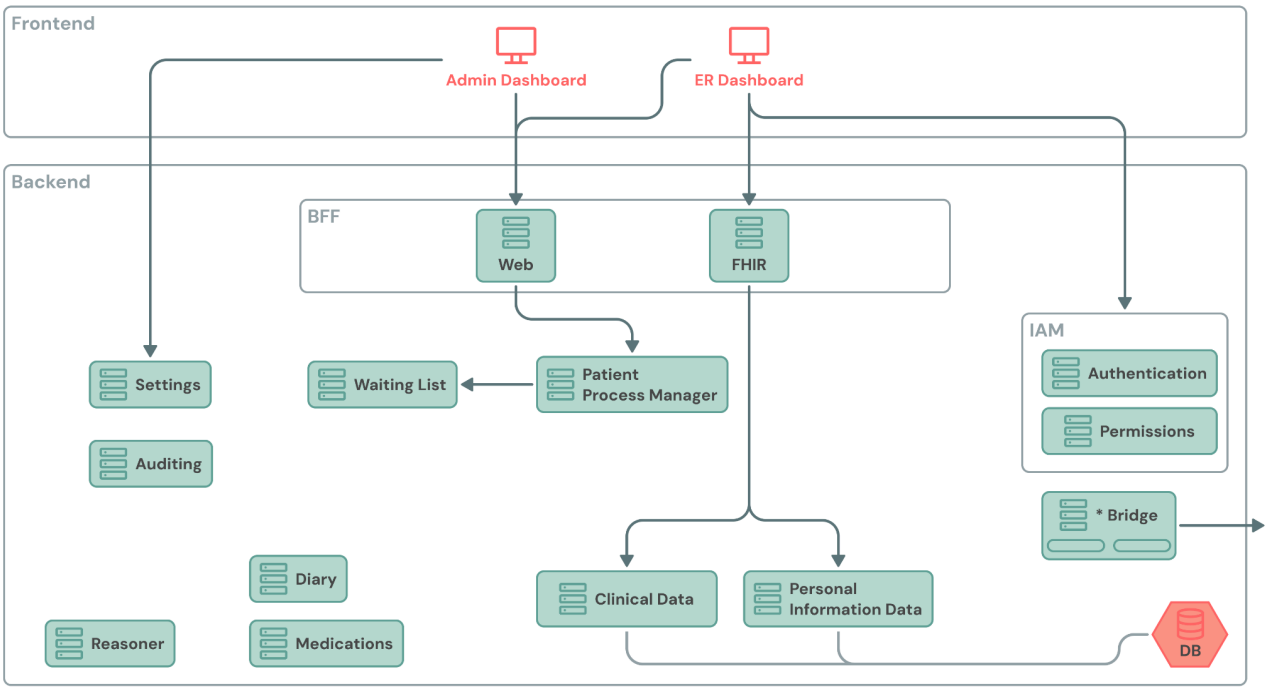


Figure 1. Architecture overview of the eCREAM-EHR.

Frontend

This layer consists of two web applications:

- an *admin dashboard* will enable system administrators to manage EHR custom configurations, user permissions, and general settings related to the platform's operations (e.g., which clinical examination approach is used among the supported ones)
- an *ER dashboard*, which represents the core web application intended for ER professionals. It will assist them in managing patient data, tracking their status on the waiting list, and overseeing the clinical operational aspects of the ER.

The applications belonging to the frontend will either communicate directly with backend services or via a *Backend for Frontend* (BFF) component. The BFF acts as an intermediary, providing a streamlined and well-focused interface tailored for each frontend module, thus reducing the complexity in the frontend layer.

Backend

The backend layer consists of multiple service modules designed to handle specific business logic and data management tasks. Among these are *core* services, which provide essential and cross-cutting functionalities for the platform. These are the EHR building blocks and include components for user authentication, permission management, and configuration settings.

In addition to the core services, there are specialized modules responsible for orchestrating *patient flow* and tracking the patient's status throughout their stay in the ER. These services ensure that the patient's journey is monitored in real-time, from initial check-in to discharge. Critical data, such as medical records and patient identification information, are managed and securely stored by dedicated services.

Given the diverse scenarios in which the EHR application may be deployed, additional services could be necessary to ensure seamless functionality. These services might be required to support specialized features, enhance interoperability, or enable smooth integration with other information systems, such as laboratory management or billing.

Each service will be explored in more detail in the next section of this document.

Component Definition and Interaction

The interaction between modules within the system will primarily rely on HTTP calls. Each module will expose a set of REST-like endpoints to facilitate synchronous communication. However, the architecture could also support event-driven communication to accommodate scenarios requiring asynchronous processing. In such cases, modules can broadcast event messages, allowing other services that have an interest in these events to subscribe and react accordingly.

External Integration and API Exposure

In addition to internal communication, the platform's HTTP APIs will be accessible to external services, extending the system's interoperability. The platform will also expose RESTful API compliant with the FHIR[1] (Fast Healthcare Interoperability Resources) specification, making it suitable for healthcare data exchanges. This API is designed to be consumed by external systems, such as the data extraction tool developed by the eCREAM partner Astir, to facilitate seamless data sharing and integration.

Security and Authentication Mechanisms

All external API calls will be secured using OpenID Connect[2] (OIDC) for authentication. This ensures that only authenticated users or trusted external services can access the platform's resources. Permissions will also be verified for every incoming request.

Mutual TLS (Mutual Transport Layer Security or mTLS) will be implemented for internal communication between microservices to ensure a secure channel. Additionally, to address the deputy problem — where a service acts on behalf of a user — the system will propagate context information that includes the identity of the user initiating the request. This allows downstream services to act in the correct user context while maintaining security and traceability.

The system will enforce strict access control on every incoming request, whether it comes from internal or external services. Each service will verify that the user initiating the request has the appropriate permissions to access the requested resources. This is essential for maintaining compliance with security policies and protecting sensitive data from unauthorized access. Access control mechanisms will integrate with the platform's identity and permission management systems to dynamically evaluate permissions based on user roles and resource access policies.

[1] https://hl7.org/fhir/
[2] https://openid.net/developers/how-connect-works/

Platform Modules

The platform has been structurally divided into several distinct modules. This preliminary design may evolve during the early stages of development based on the new requirements being gathered. These modules can be divided into three main layers: frontend, BFF and backend.

The frontend layer includes two primary web-based user interfaces: the *admin* and *ER dashboards*. These dashboards are tailored for specific user roles and provide critical functionality for both administrative and ER operations.

Admin dashboard

The admin dashboard is a web application designed for administrative users, enabling them to manage global settings, configure user accounts and their respective permissions, and modify system-wide configurations. It acts as the central control panel for system administrators to perform all the system management tasks.

ER dashboard

The ER dashboard is the primary interface for healthcare professionals in the emergency room. It provides access to relevant data, such as patient information, clinical records, and monitoring tools necessary for efficient ER management. Key functionalities include:

- monitoring the waiting list and patient flow through the various ER stages
- viewing and updating clinical data for individual patients
- supporting emergency room workflows with relevant clinical tools and interfaces

Both dashboards will communicate with the backend services via HTTP APIs, enabling operations such as data retrieval, user management and clinical workflows. Applications will interact with these backend services either directly through exposed service endpoints or via Backend for Frontend (BFF) services. The BFF services act as intermediaries between frontend applications and the backend, offering optimized data aggregation and computation layers for the dashboards.

Web API

The Web API is a dedicated backend service designed to handle requests from web-based dashboards. Having a dedicated BFF module brings some advantages to the platform. In particular, the Web API module can serve tailored APIs to each dashboard, ensuring the frontend only fetches the data it needs. At the same time, it improves the system performance by aggregating data from multiple backend services, minimizing the number of calls the frontend has to make.

FHIR API

The FHIR API is a specialized service that provides FHIR-compliant endpoints. It facilitates the exchange of medical and clinical data with external systems, such as third-party healthcare providers or other platforms requiring access to patient and clinical information. The FHIR API

module ensures that all data exchanges adhere to industry interoperability standards, promoting seamless standardized communication across various systems, such as the data extraction tool, which is part of the eCREAM project (WP2).

The backend layer encapsulates the core business logic, data management, and platform workflows. It consists of several sub-layers and services, generally categorized into core services, process management services, data storage services and other microservices with specific functionality. Depending on whether a specific request is coming from the frontend applications or other clients, one or more backend services may be invoked to process the request and collaborate to provide the necessary response. It follows a non-exhaustive list of backend services that will be implemented.

Authentication and Permissions (Core)

The Authentication and Permissions services are fundamental components responsible for managing user identity, access control and resource protection. Together, they ensure that users can access only the functionalities and data appropriate to their roles within the platform. With respect to Figure 1, they represent the IAM (Identify and Access Management) box.

The authentication module is responsible for handling the user login phase and is involved in the subsequent incoming requests to the backend services. In order to perform requests to the backend, the clients must own a valid token issued by the Authentication module. Tokens can be obtained by the user who logs in. In addition, the module supports machine-to-machine authentication to enable an external system to access the available platform APIs.

The user's authentication mechanism is implemented following the principles of the OIDC protocol, ensuring a standardized and secure method for verifying user identity. Upon successful authentication by an external Identity Provider (IdP), the authentication module, which acts as a Relying Party server (RP), issues a JSON Web Token (JWT) to the user.

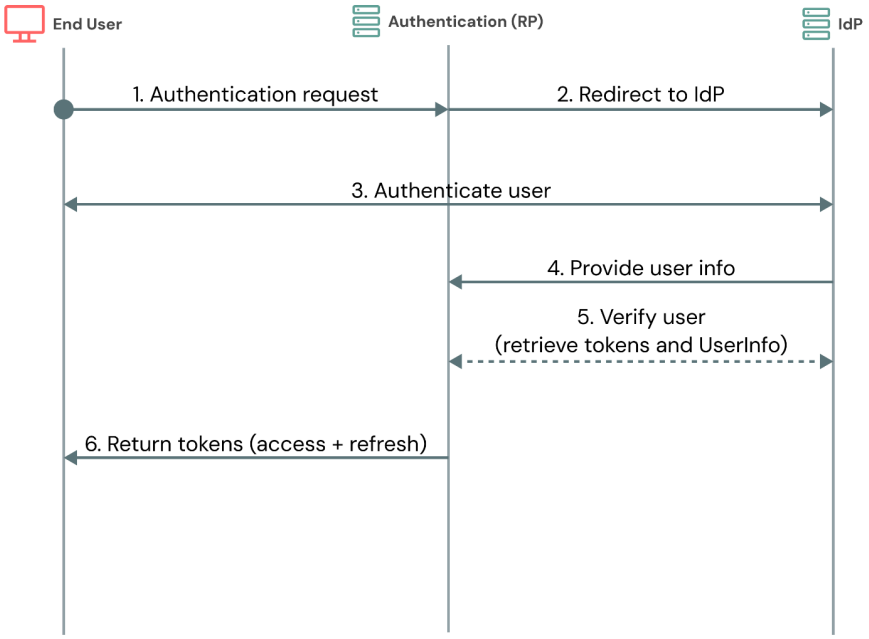


Figure 2. Login workflow. End User, Authentication module and an IdP are involved in the full process of login.

In a typical workflow (see Figure 2), the user who wants to log in to the ER dashboard (or another eCREAM application) is initially redirected by the system to the configured Identity Provider's login page. After successfully logging in, the IdP provides the Authentication module with the information needed to verify the user's identity. Subsequently, the module can release the authentication credentials to the user (i.e., the ER dashboard), which consist of an *access token* and a *refresh token*. Decoupling these new tokens and the credentials released by the IdP allows the platform to be more flexible and potentially integrable with custom, already existing, hospital authentication services. Once the client application owns the token, it can present it to the platform's backend services to assert the user's identity. As described in Figure 3, the client requests a specific platform service, including the access token in the HTTP call. The backend service, in turn, validates the token with the JWKS (JSON Web Key Set) keys published by the Authentication module that have been used to sign the issued tokens.

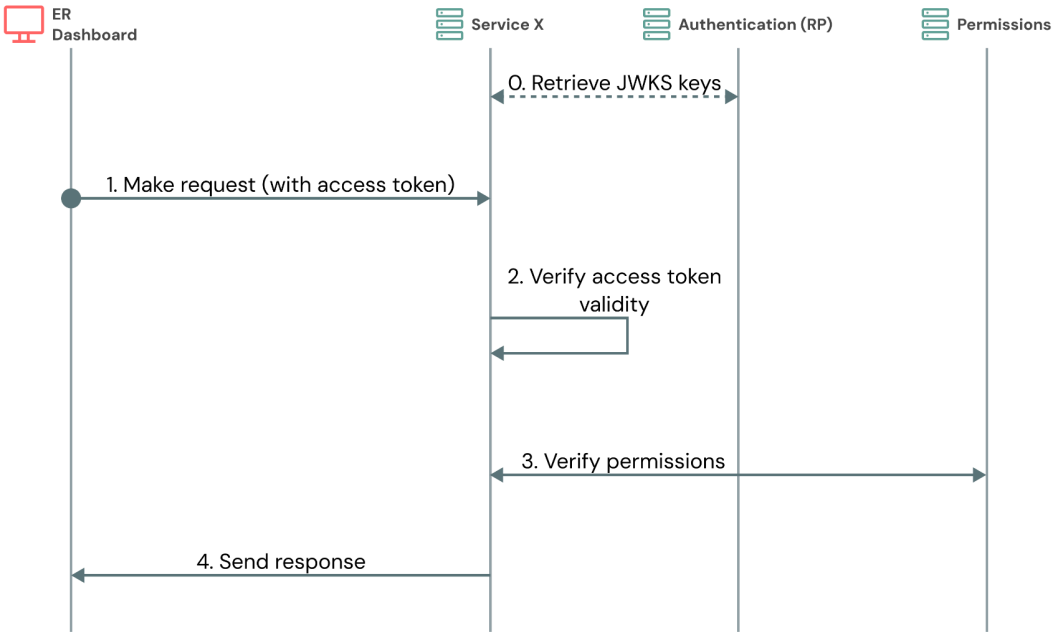


Figure 3. A typical client's request workflow.

Once user identity is confirmed, the platform proceeds with permission verification to determine whether the authenticated user is authorized to perform specific actions or access certain resources. Authorization checks are handled by the Permissions service, which is built using Casbin, a powerful and efficient open-source access control library. The Permissions module employs an Attribute Based Access Control (ABAC) model, allowing administrators to define access policies and rules that regulate user interactions with platform resources.

This decoupled approach to authentication and authorization promotes security and enhances the system's scalability and maintainability. Nonetheless, it also ensures flexibility, as the system can be integrated with different Identity Providers and can be configured without requiring deep changes to the authentication business logic itself.

Machine-to-machine (M2M) authentication implementation follows the OAuth pattern. The client will be provided with Client ID and Client Secret. These credentials can be used to perform the authentication and obtain the access token. This token grants access to the protected resources and endpoints of the platform's services.

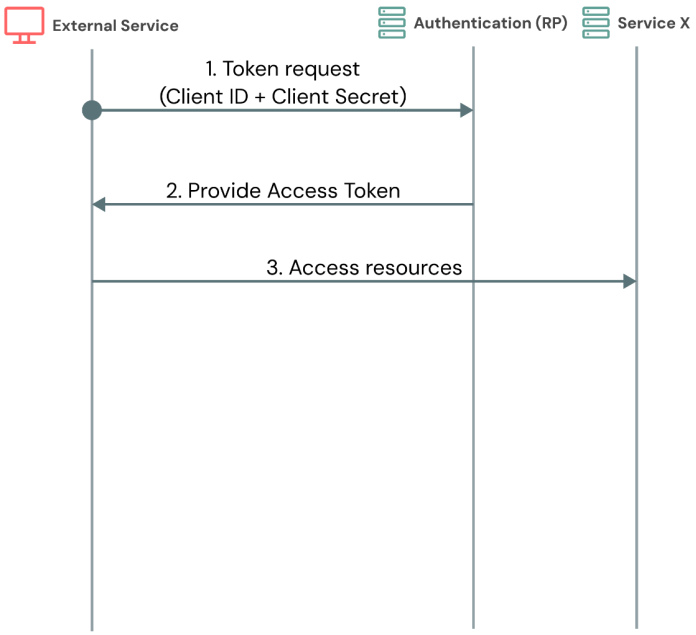


Figure 4. M2M Authentication Workflow.

Settings (Core)

The Settings module is responsible for managing both global system configurations and user-specific preferences, providing a flexible and customizable experience within the ER platform. For example, global settings control the activation or deactivation of specific sections and tools across the ER dashboard, ensuring that administrators can tailor the platform's functionality to meet the organizational needs of the hospital where the platform is deployed. This capability represents a crucial requirement since the eCREAM project requires the EHR platform to be deployed in different scenarios (and, potentially, different countries).

User preferences, on the other hand, allow individual customization of various elements within the application, including data tables, filters, input forms, interface language and visual components. This ensures that healthcare professionals can personalize their interaction with the platform to suit their workflows and preferences (e.g., allowing a physician to save a list of preferred therapies).

During focus meetings, healthcare professionals emphasized the demand for deep customization of the ER dashboard, highlighting the importance of adapting the platform to fit diverse clinical practices and operational requirements. Nevertheless, the application will maintain a consistent set of core functionalities and data structures across the different local instances in order to preserve the overall homogeneity of the system for comparability of data in future emergency medicine research.

Auditing (Core)

The Auditing module provides essential functionality for tracking and recording operations performed by users and relevant system-wide activities. The other modules within the platform can send audit information through the API exposed by this service. The collected information is stored in a dedicated database and, all together, represents a detailed history of what occurred in the system at any given time. This service enables comprehensive monitoring and can support compliance, security, and troubleshooting efforts.

Patient Process Manager and Waiting List (Process)

The Patient Process Manager and Waiting List modules are responsible for managing and tracking the entire patient journey throughout their stay in the emergency room. These services manage the state of each patient, from registration to discharge, ensuring that all process stages are captured and monitored on demand.

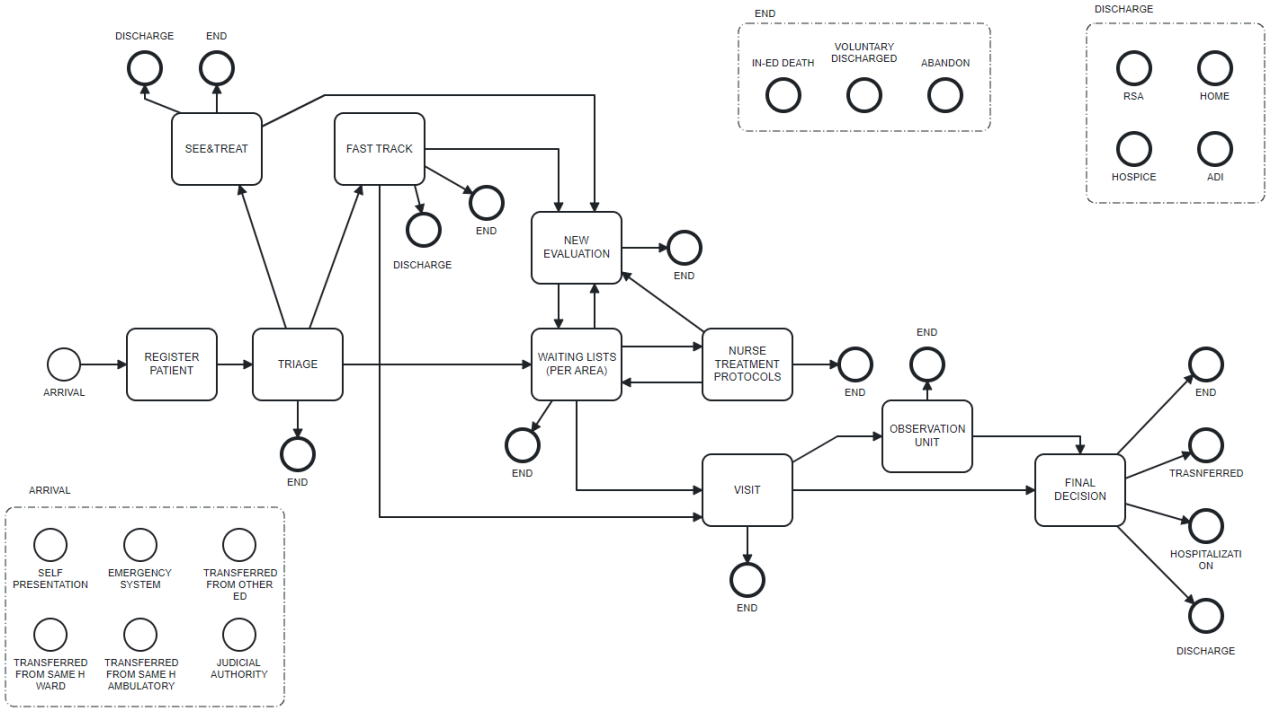


Figure 5. Patient workflow modeled using BPMN.

Figure 5 describes the flow of a patient's data, to model the actions that the patient takes or is subjected to. Each rectangle represents a stage (i.e., the set of actions performed at a certain point in the process). A circle represents the starting point; a circle with a thicker border identifies the exit point. Since a patient may enter or leave for many reasons, boxes with dashed lines summarize the possible options. Not all the participants of the eCREAM project may have, process, or execute the stages shown in the flowchart. Each stage, however, has been deemed either of common use to at least one of the participants, or is relevant to most of them. To accomplish this degree of flexibility, it will be necessary to set up these microservices with the desired configuration when the application is initially deployed.

The Patient Process Manager handles the creation of new process instances for each patient, tracking their progression through various stages such as triage, evaluation, treatment, and discharge. It supports complex workflow management, including the patients' handover, the shift rotations of clinicians, and the implementation of the different clinical approaches (e.g., AMPLE, ISBAR) that may be adopted in the context of emergency medicine.

The Waiting List module should provide a global view of the ER's current status, displaying the total number of patients and raising alerts when patients have been waiting too long. Furthermore, it should allow a physician to move a patient to a different waiting list, should the condition of a patient change. The implementation of the Waiting List module will vary depending on the hospital in which it is deployed. This is because different hospitals may employ different types of waiting lists. These include the canonical emergency codes, the intensity of care required by the patient, or hybrid approaches.

Personal Information Data (Store)

The Personal Information Data module stores and provides patients' personal information to the modules or clients requiring it. Data includes identity information, contact details, and demographic data. The module could also include personal information about the other users (healthcare professionals) of the platform, if deemed relevant to the platform's functionality.

Data is stored in a dedicated database of this service, available only through the provided APIs, and access is subject to prior permission verification.

Clinical Data, Medications, and Diary (Store)

The Clinical Data, Medications, and Diary services are responsible for storing and managing key patient clinical information, including observations, medical histories, treatment plans, prescriptions, and clinical diary entries. These services ensure that healthcare providers can access accurate and comprehensive clinical data for effective patient care.

Data is modeled and exchanged using the FHIR standard, providing integration and communication with external systems and the platform's internal web applications. Clinical data is also stored in the database using the proper FHIR resource structure to maintain interoperability and consistency.

Though closely related to clinical data, the Medications service is implemented as a separate module due to the complexity of treatment management. It goes beyond simple storage, offering additional functionalities like maintaining drug lists, performing medication validation, providing basic prescription suggestions, identifying common prescribing patterns, and controlling adherence to clinical guidelines.

Similarly, the Diary service has been separated from the clinical data storage because it addresses a well-identified, specific functionality of the platform: collecting additional clinical information about the patient after the first clinical examination. In addition to the classical read-and-write operations on diary textual entries, it provides additional capabilities.

Following consultations with experts, two modes of interaction with the clinical diary were identified: direct and indirect. In the former case, the user can edit and update information in the diary manually. This may include entering new clinical data, re-evaluating the patient's status, updating the therapeutic strategy, and viewing examination results. It is essential that this interaction be expeditious and intuitive. The second option entails automatically integrating and updating data from other digital medical record pages (e.g., examinations, therapies, vital parameters) into the clinical diary. This approach is driven by the necessity to include a more comprehensive range of information when exporting and sharing the patient's status with other professionals and relatives.

Reasoner (Process)

The reasoner module provides decision support by processing clinical data to offer suggestions, alerts, or insights to healthcare professionals. The reasoning engine analyzes patient data and exploits clinical guidelines and other domain knowledge bases to recommend potential actions or treatments, highlight data inconsistencies or logical incongruences, signal multi-drug interactions, etc.

The more detailed design and definition of the module will be part of task 3.5, which started in September 2024. Meanwhile, examples of its usage have already been collected during the focus meetings with the healthcare professionals involved in requirements definition and user interface design.

Bridges (Process)

The Bridges modules are designed as auxiliary components of the ER platform to enable integration with various external hospital information systems. Each of these modules offers a set of functionalities allowing the EHR application to interact with hospital services such as laboratory exams (Exams Bridge), billing (Billing Bridge), patient records, and other clinical and administrative data.

Modules are structured to support scalability and future expansion. Their design allows for ongoing development, ensuring they can accommodate new functionalities depending on the future requirements and the context in which the EHR is deployed. To ensure compatibility with different hospital IT environments, multiple implementations of a certain module will be provided. For instance, the Exam Bridge will facilitate access to laboratory exams and enable operations such as ordering lab tests, checking their status, and consulting their outcomes. Given that each hospital may use a unique LIS (Laboratory Information System), the bridge module must implement specific data formats and communication protocols to integrate with these third-party systems.

Each implementation will be tailored to align with the custom requirements of different external systems, adapting to the unique technical characteristics, security protocols, and integration standards of different hospitals. At the same time, the interface specification and the communication protocol used within the platform and available to the other modules will remain unchanged. The use of a dedicated module minimizes external dependencies,

establishes a clear separation between the ER platform's internal components and the hospital's systems, and eliminates the need for extensive rewrites during future upgrades or updates. This modular approach ensures that changes to either the hospital systems or the ER platform can be managed independently, fostering maintainability and adaptability over time.

As the platform evolves, additional services may be developed based on new requirements and use cases that arise. The modular design of the backend allows for easy integration of new functionalities without disrupting the core system architecture.

Software Stack

The development team has recently completed the initial research phase, assessing which principles, technologies, and programming languages are most suitable for building the EHR software. This includes evaluating various database technologies, UI frameworks, and development languages to ensure they align with the project's scalability, security, and performance goals. By thoroughly exploring these options, the team aims to make well-informed choices that will establish a strong foundation for the long-term success of the EHR platform.

The rest of this paragraph is dedicated to a presentation of the technology stack that is likely to be used during the development of the platform. However, it is worth noting that changes in the adopted frameworks and technologies may happen in the future, according to the evolution of the project and to tackle possible changes in the functional requirements of the EHR system.

The chosen frameworks and technologies for the EHR platform are carefully selected from the most widely adopted and cutting-edge options available in the open-source community. Priority is given to technologies that are actively maintained, well-documented, and backed by strong developer communities to ensure long-term support and continuous improvements. Security is also critical, as we aim to implement solutions with proven security features and best practices. By relying on these industry-leading tools, the development team ensures that the platform remains modern, reliable, and capable of adapting to future demands.

Frontend

The frontend layer will be mainly based on the following technologies:

- Vue.js, *an approachable, performant and versatile framework for building web user interfaces*
- Nuxt, *an open-source framework that makes web development intuitive and powerful*
- Nuxt UI *simplifies the creation of stunning and responsive web applications with its comprehensive collection of fully styled and customizable UI components designed for Nuxt*
- Tailwind CSS, *a utility-first CSS framework*
- Node.js, *a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.*

Backend

The backend layer will be mainly based on the following technologies:

- Fastify, *a web framework highly focused on providing the best developer experience with the least overhead and a powerful plugin architecture*
- PostgreSQL, *an innovative data management system known for its reliability, robustness, and extensibility*

Each component of the architecture will also be provided as a container to simplify the deployment of the platform in local or cloud clusters based on container orchestration tools, like Kubernetes.

Storage

PostgreSQL stands out as a highly versatile and robust open-source database solution, making it particularly well-suited for developing EHR systems. One of its key strengths is its ability to handle both *structured* and *semi-structured* data seamlessly, thanks to its support for JSON. This flexibility allows developers to manage a wide range of healthcare data types, from highly structured patient records to more flexible formats like clinical notes, all within a single platform.

User Interface Prototype

In conjunction with the design of the architecture of the eCREAM-EHR, the design of the user interface of the ER dashboard started as defined in task 3.4. More specifically, the development team, along with a UI/UX designer, participated in several focus group discussions and meetings with healthcare professionals from various Italian hospitals.

The aim of this task is to provide screen flows that align with the flow of information and the real-world procedures defined in task 3.2, ensuring smooth and consistent user-system interaction. The result is a high-fidelity prototype developed in Figma, designed to fulfil two critical functions.

First, it provides a graphical representation of the eCREAM-EHR, offering a detailed, visually accurate depiction of the user interface. This allows stakeholders, such as healthcare professionals and project managers of the eCREAM partners, to see and evaluate the design in a form that closely resembles the final product. By simulating the look and feel of the actual dashboard, the prototype enables early feedback on usability, layout, and visual hierarchy, helping to refine the design before development begins.

Second, the prototype serves as a blueprint for developers during the implementation of the user interface. It offers clear guidance on how each dashboard element should appear and behave, including interactions, animations, and transitions. The high level of detail in the prototype helps bridge the gap between design and development, reducing ambiguity and guaranteeing that the UI is built according to the intended user experience.

User Experience (UX)

To make the platform fully aligned with User eXperience requirements, the criteria we focused on are usability, efficiency, and effectiveness. These criteria shape the design and the iterative refinement of the platform, ensuring rapid and accurate handling of patient data.

Usability means how effectively the platform supports the needs of medical staff in the ER. It involves providing quick, intuitive access to critical patient information and tools, ensuring that features are easy to navigate and understand. Moreover, it ensures that the platform reduces cognitive load by presenting relevant information clearly and minimizing the risk of errors through validation and feedback[3].

Efficiency focuses on enabling medical staff to perform tasks quickly through the simplest and most direct processes. The platform allows easy access to crucial functions, such as visualizing patient records, ordering tests, or updating treatment plans, which can be completed in the fewest possible clicks, without unnecessary steps.

[3] Interaction Design Foundation — IxDF. (2016, June 1). What is Usability?. Interaction Design Foundation — IxDF. https://www.interaction-design.org/literature/topics/usability

Effectiveness intends to support users in completing tasks accurately. By providing clear prompts, error-prevention mechanisms, and real-time feedback, the system wants users to confidently enter and retrieve data with minimal risk of mistakes.

User Interface (UI)

To guarantee the fulfilment of UX criteria, the platform has been designed as a Single Page Application (SPA). This architectural choice ensures that users can access essential features and data quickly, reducing cognitive load and improving task efficiency — key elements in emergency medicine settings. In addition, the platform is designed to make sure that critical data is visible without scrolling. Information is organized into modular sections that can be expanded as needed. This prevents users from having to scroll through long and cluttered screens, helping them focus on the most pertinent information.



Figure 6. Prototype layout.

We identified a structured layout with three key components to improve the user experience: the **sidebar**, the **main content area**, and the **patient overview**. Each component plays a crucial role in making the system intuitive and responsive to the dynamic needs of the ER.

Sidebar

It acts as the primary navigation tool (menu), allowing users to quickly move between different sections or pages of the platform. This component is designed to change dynamically based on the current task or context, highlighting the most relevant options and making them always easy to reach and visible when selected. Furthermore, its fixed position on the interface

ensures that users can always return to previous sections or navigate to new ones without losing their focus on the system.

Main Content Area

This is the area where users perform most of the actions provided by the platform, first and foremost, the recording of patient information. In order to ensure systematicity, completeness and quality in data collection, this section is highly structured. Thus, information is entered mainly through clicks on items predetermined by the clinical study team. This entry mode reduces registration time and ensures the highest level of comparability and interoperability of data collected by different operators in different EDs, greatly increasing their usability in research settings. To maintain the readability of the folder, a function will be implemented that can translate each click into a full-text sentence that automatically appears in the box below the area. In this box, it will also be possible for the operator to add free text to supplement the structured information with more clinical details when needed.

This section is divided into smaller subsections that are clearly labelled to enhance clarity and ease of use. Users can navigate through patient records, treatment plans, diagnostics, and medical history in a well-organized format, improving task accuracy.

To simplify workflows and support task efficiency, the content area incorporates pop-up overlays for more detailed data entry, such as filling out forms or inputting diagnostic details. These pop-ups assist users, who can add or modify information without leaving the main content area.

Patient overview:

It serves as a quick reference for the current patient's general status and key clinical information. Positioned on the right for easy access, this component displays the patient's basic information, such as name, age, and current location within the ER alongside a visual, human-like figure, the *homunculus*. This representation provides a snapshot of the patient's condition, visually representing relevant clinical data, such as trauma locations, employing icons overlaid on a human figure to denote specific injury sites. The severity of these injuries is indicated by colour-coding (still under analysis and definition) and offers a rapid and intuitive understanding of the patient's overall status. For example, a red icon on a wrist might signify a severe fracture, while a yellow icon on the chest could indicate moderate respiratory distress. This visual reporting system allows healthcare providers to recall patient's clinical conditions immediately. It is a crucial feature in the emergency room setting, where providers are simultaneously in charge of an average of 5 to 15 patients. Under these circumstances, the physician and the nurse very often shift their attention from one patient to another, with an inevitable increase in errors. A system that allows at-a-glance recognition of the patient and his or her clinical conditions greatly reduces errors or near-misses, increasing operators' confidence.

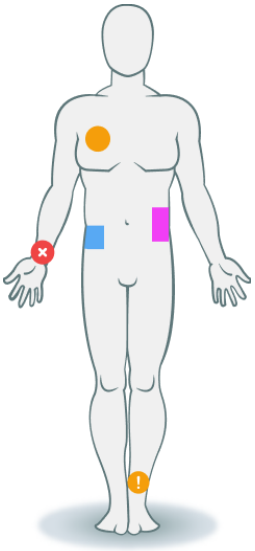


Figure 7. Example of the homunculus with relevant clinical data.

The overview is further enhanced by data regarding vital parameters and ABG, making the process of monitoring health metrics always available and therefore reflects on the option to make faster and informed decisions.

The Prototype

In this paragraph, we describe in detail the work done on the prototype of the ER dashboard. In particular, we first summarize all the pages belonging to the layout. Successively, we offer an overview of some of the pages that allow healthcare professionals to manage patients throughout their stay in the emergency department. It is worth noting that the current version of the prototype is still under development, and subject to changes and improvements.

Recently, we decided to update the dashboard style by introducing a new design system aligned with the Nuxt UI framework. This decision was made to achieve a more consistent and modern aesthetic that enhances usability and accessibility across the platform. To increase visual consistency, the design system introduces a unified set of guidelines for colours, typography, icons, and components.

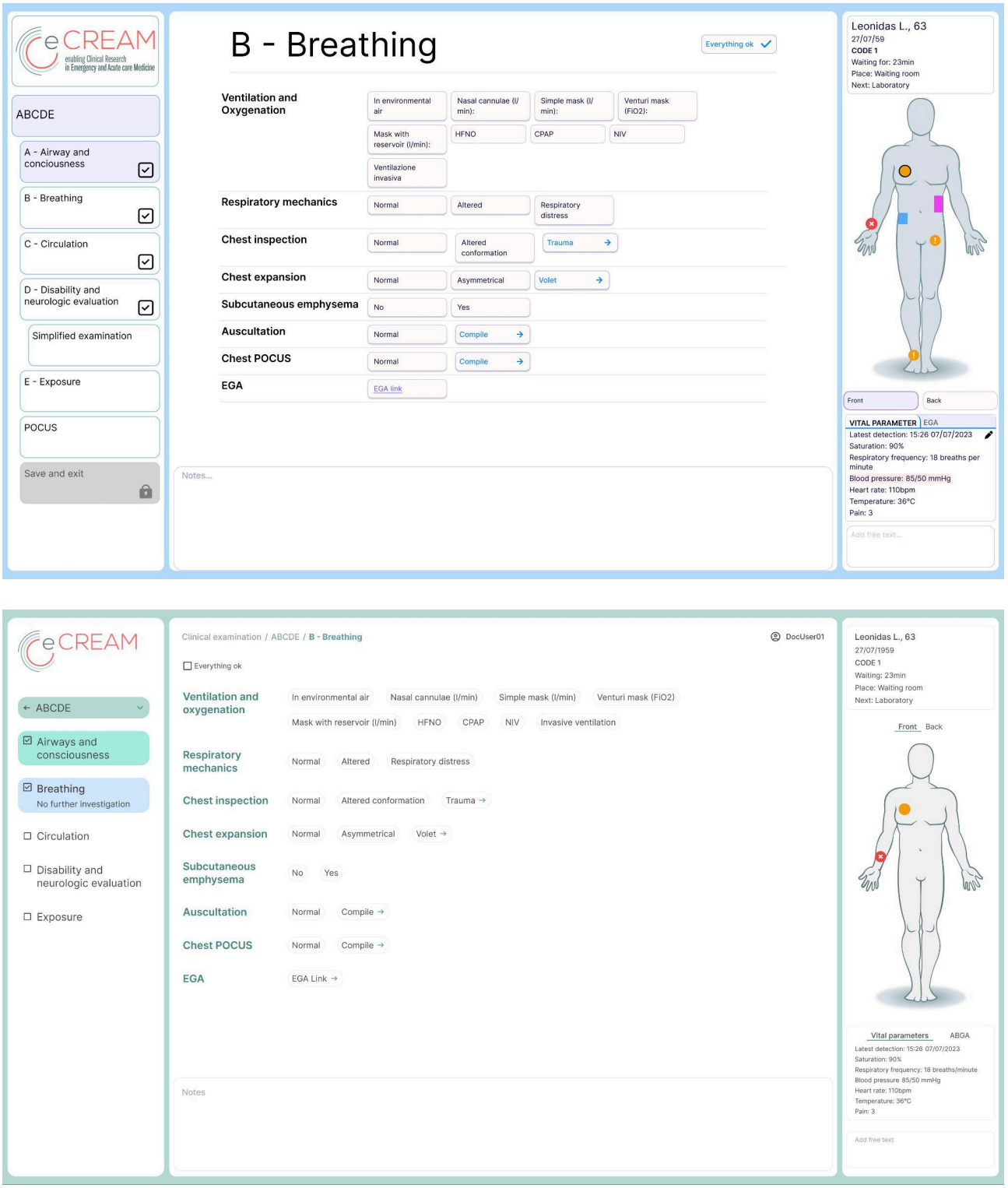


Figure 8. Example of the page Breathing (Clinical examination) before and after the restyling of the platform.

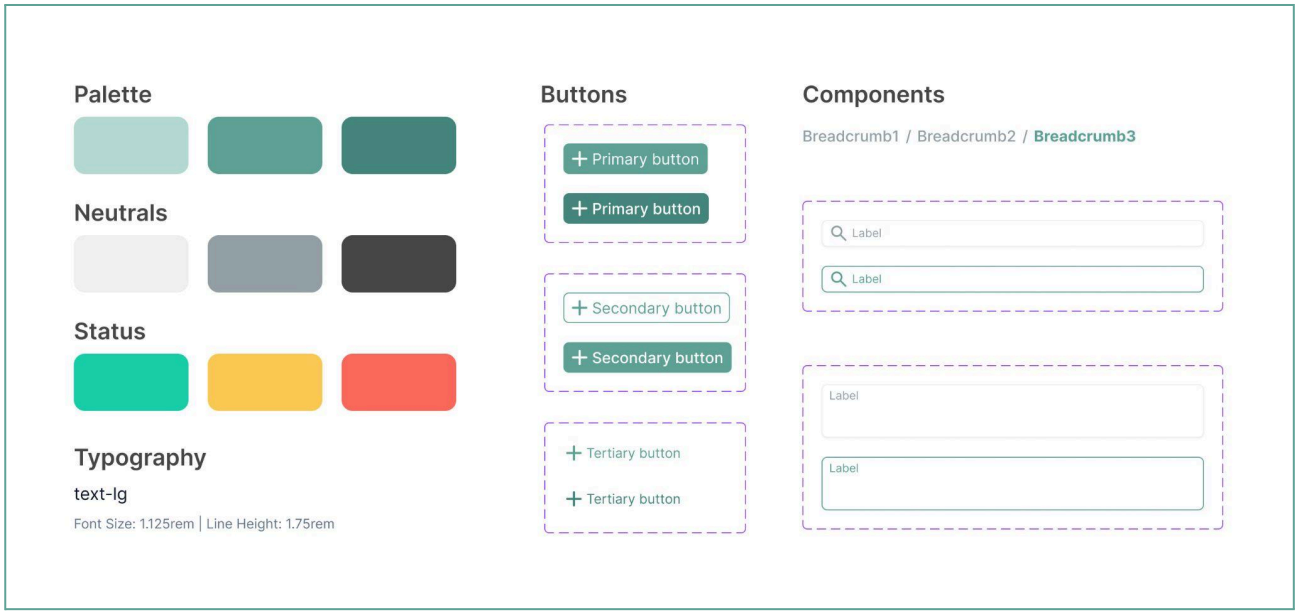


Figure 9. Design system.

In collaboration with domain experts, we devised a page structure comprising a series of functional units to facilitate the compartmentalization of patient-related functionalities. To support consistency in research quality, these pages serve as a standardized framework, representing the agreed-upon core aspects of patient care. In other words, all the installations of the EHR platform will present the same sections. Nevertheless, the system administrators will be allowed to customize each of them in accordance with the local standard procedures adopted by each hospital organization and legal health regulations.

The following is a list of the identified pages with a brief description. When available, screenshots of the prototype are included to offer an insight into the visual representation.

Triage

In this page, users can quickly assess incoming patients' conditions, assign priority levels based on severity or intensity of care, and document key observations. This page enables efficient initial assessments and helps organize patient flow in the ER. The context analysis revealed that the adopted modalities vary significantly across different hospitals. Consequently, while the ER dashboard may serve as a default methodology for triage, customized solutions will be implemented to meet each hospital's specific requirements.

Vital Parameters

On this page, users can record, monitor, and review patients' key vital signs — such as heart rate, blood pressure, temperature, and oxygen saturation. The page provides a clear overview of critical health indicators, allowing for continuous tracking and prompt detection of any change in the patient's condition. In addition, as described previously, the latest updates of vital parameters will be available to clinicians in the patient overview section.

Anamnesis

The page allows users to document a patient's medical history following the AMPLE approach, which includes Allergies, Medications, Past medical history, Last oral intake (Last meal), and Environment leading up to the current condition. This approach ensures that all relevant background information is gathered efficiently, providing a comprehensive view of the patient's health history to inform diagnosis and treatment.

For example, in the Allergies subsection, users can enter detailed information about a patient's allergies. Single-click buttons allow them to quickly specify "no allergies" or "unknown allergies" if applicable. Users can also search the database for common allergens or manually add specific allergies, such as reactions to contrast media and other known allergens. This intuitive interface simplifies data entry while ensuring comprehensive allergy documentation.

In addition, integrating the reasoner module will enable the system to provide contextually relevant recommendations based on previously collected information about the patient. For instance, the system could alert users to potential drug interactions or contraindications based on the patient's medical history, such as allergies or existing conditions. This topic will be deeply investigated in the next deliverable (3.3), First release of the reasoner module.

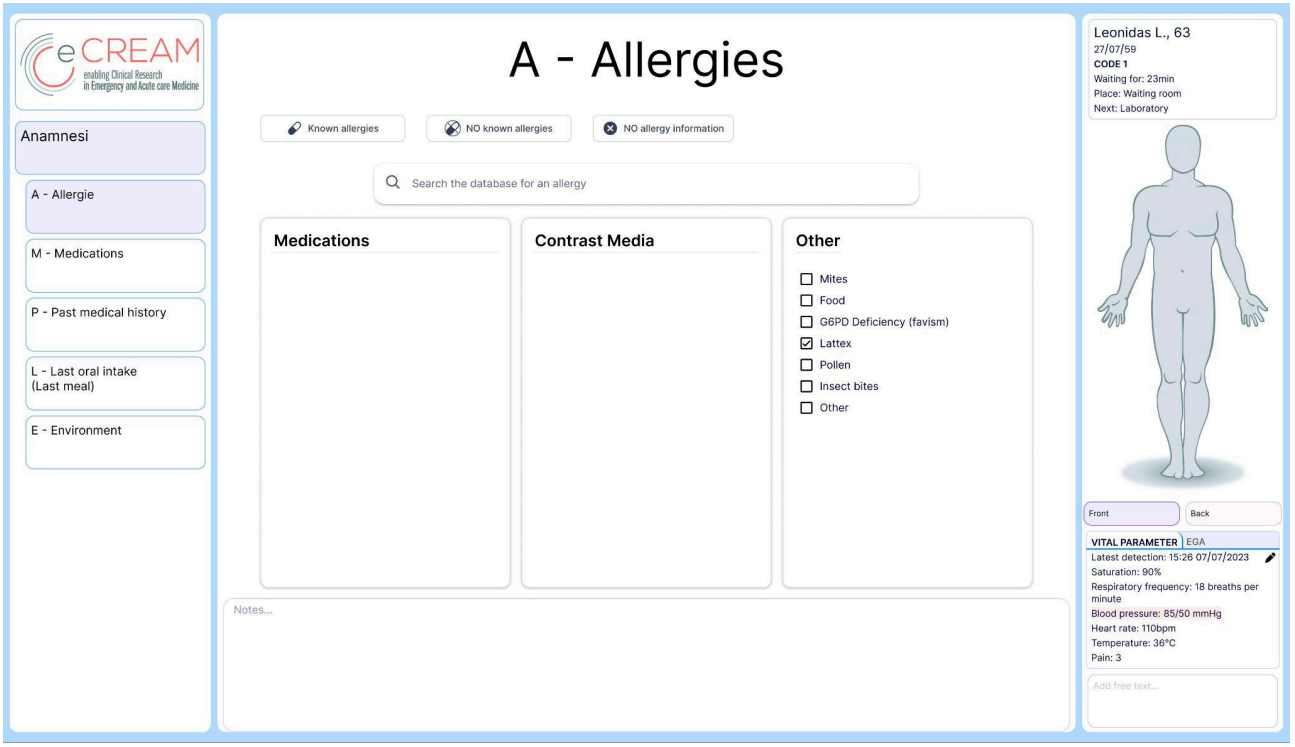


Figure 10. Screenshot of the page Allergies (Anamnesis).

Clinical Examination

The page enables users to conduct and document a thorough physical assessment of the patient, utilizing frameworks such as ABCDE (Airway, Breathing, Circulation, Disability, Exposure), Head-Toes assessment, or examination by apparatuses. Regardless of the chosen

approach, users collect the same clinical information, ensuring a comprehensive evaluation of the patient's condition while presenting the data in a format that best suits their clinical practice.

Depending on the patient's condition and its severity, the physician decides what to assess and which subsections to access. To facilitate completion of the clinical examination section, a shortcut has been added to mark a subsection as not clinically relevant (with an "Everything OK" button). In other words, clicking this button means that the physician claims to have examined all the items on the page, having found nothing pathological. Conversely, the user goes through the various categories of a subsection and selects the values that best represent the patient's condition. In both cases, the dashboard returns visual feedback to the user with the help of colours; for example, the subsection button will appear yellow in case of an alert for a patient with obstructed airways, meaning that one or more clinical issues are present in that section, as shown in Figure 10. Finally, at the bottom of the page, automatically compiled text will appear according to previously chosen options, to provide a narrative description of the clinical examination.

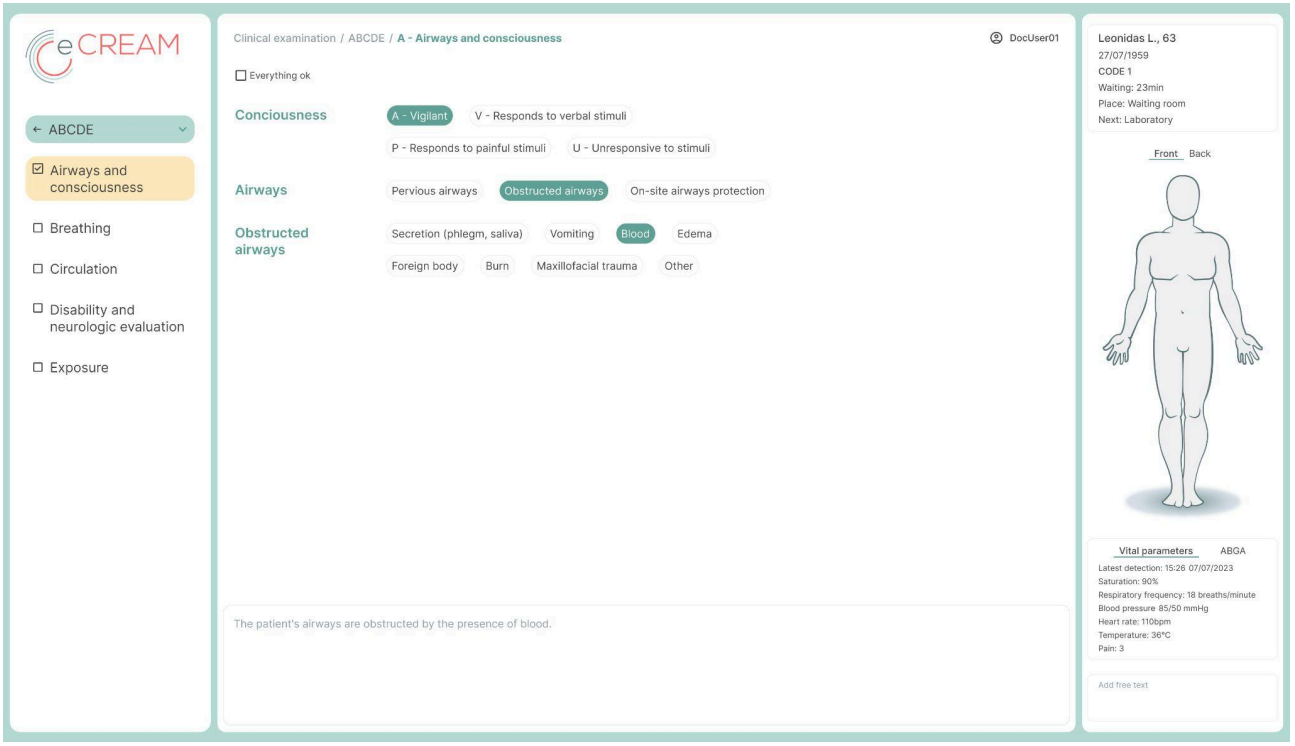


Figure 11. Screenshot of mock-up of the page Objective examination, ABCDE approach.

ISBAR

The page allows users to communicate vital patient information during handovers using the ISBAR framework (Introduction, Situation, Background, Assessment, Recommendation). This approach guarantees that essential details are conveyed clearly and efficiently, enhancing continuity of care and minimizing the risk of miscommunication during shift changes. Users can document patient status, ongoing treatments, and any concerns, ensuring that all team members are informed and prepared to provide optimal care. It is important to note that some

subsections of the ISBAR page display previously collected information; for example, the allergies collected and registered during the Anamnesis are reported in the Background section. Users, however, can always modify or provide additional clinical details as free text.

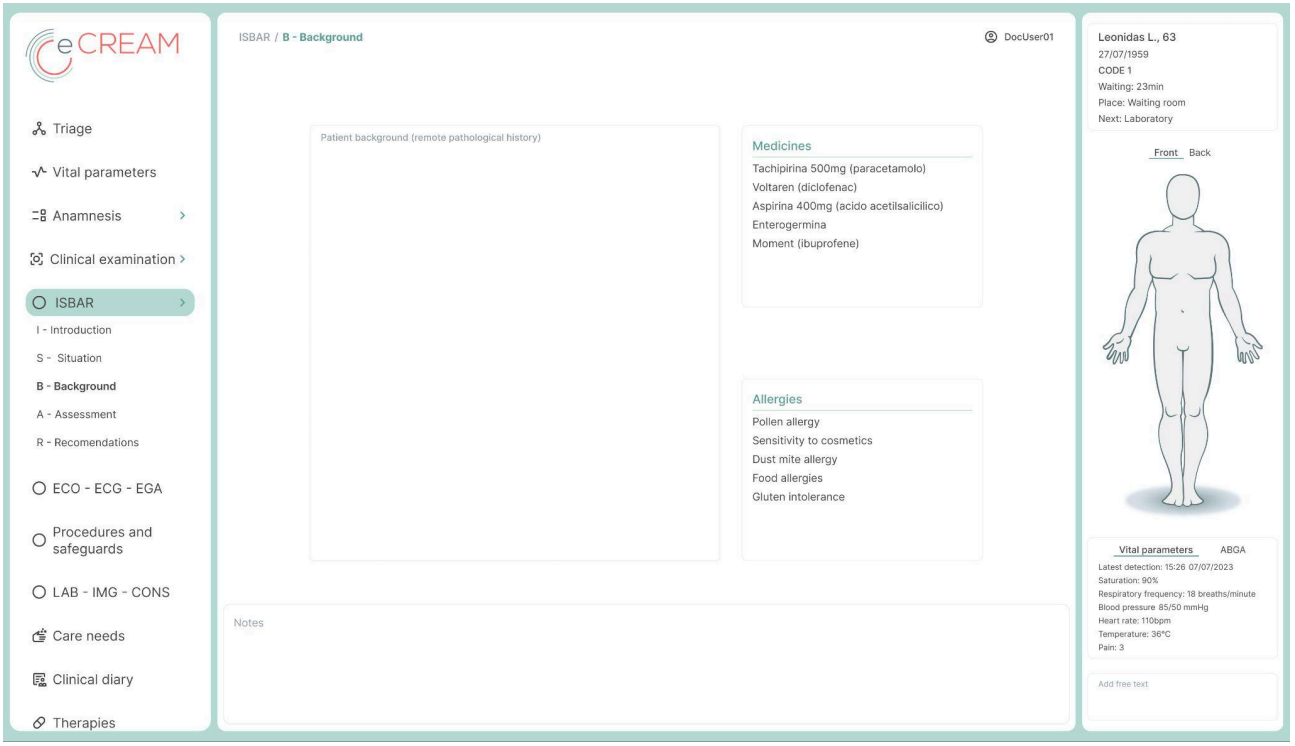


Figure 12. Screenshot of mock-up of the page Background, ISBAR framework.

ECO — ECG — ABGA

The page provides clinicians with the tools to perform essential cardiac and respiratory assessments as they can analyse Echocardiograms (ECO), Electrocardiograms (ECG), and Arterial Blood Gas analyses (ABGA).

Procedures and Safeguards

The page allows users to manage various clinical procedures performed on patients, along with the associated safety protocols. Users can outline detailed instructions for specific interventions, and ensure compliance with best practice guidelines.

LAB — IMG — CONS

The page provides interoperability with other IT systems, such as Laboratory Information Systems (LIS) and Picture Archiving and Communication Systems (PACS), allowing users to view test results and imaging data. Additionally, users can request consultations with specialists directly through the platform.

Care Needs

The page facilitates comprehensive care coordination by enabling users to prioritize and track the patient's needs, ensuring that all aspects of care are addressed effectively. Additionally, it

serves as a communication tool among the care team, allowing for updates and adjustments based on the patient's evolving condition and feedback from the healthcare team.

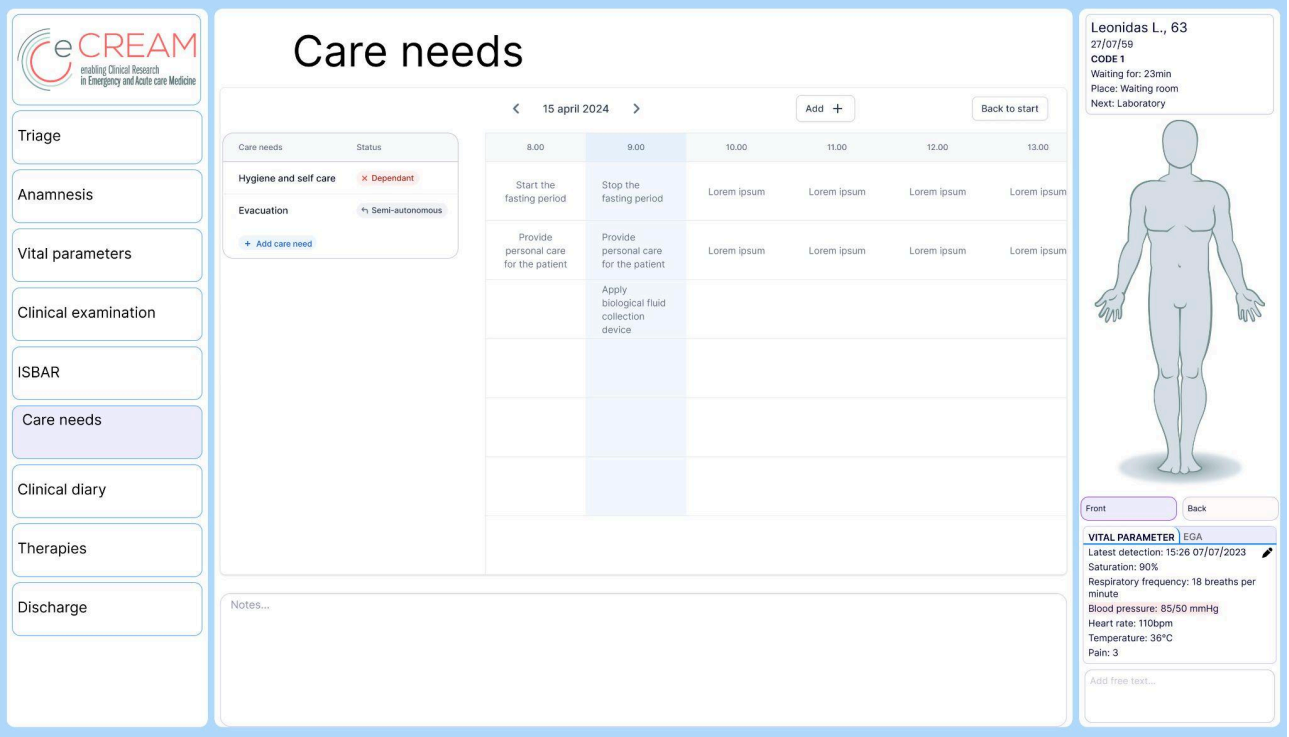


Figure 13. Screenshot of mock-up of the page Care needs.

Clinical Diary

The page provides users with a structured space to document ongoing patient observations, treatment progress, and relevant clinical notes throughout the patient's care journey. Furthermore, the clinical diary collects and displays information from other pages, such as drug or lab test prescriptions. Users can record significant events, condition changes, and treatment responses, creating a comprehensive timeline of the patient's clinical status.

Therapies

The page provides physicians and nurses with a comprehensive overview of the therapies administered to the patient and tools for tracking therapy progress and adjusting treatment plans as necessary. The calendar representation of current therapies enables clinicians to quickly understand and assess the patient's therapeutic status through colour coding and icons.

Additionally, medical staff can add four different types of therapies, i.e., extemporaneous, need-based, time-based, or continuous, by completing a well-structured form designed to enhance efficiency. Once they have selected a drug (or a combination of drugs), the chosen medication appears on the right side of the form. The staff can set the dosage and select the administration method, e.g., inhalation, rectal, or intravenous, with additional bolus or continuous infusion options. Finally, before confirming the inclusion of the therapy in the

calendar, clinicians will be asked to check whether the therapy has already been administered or is about to be administered.

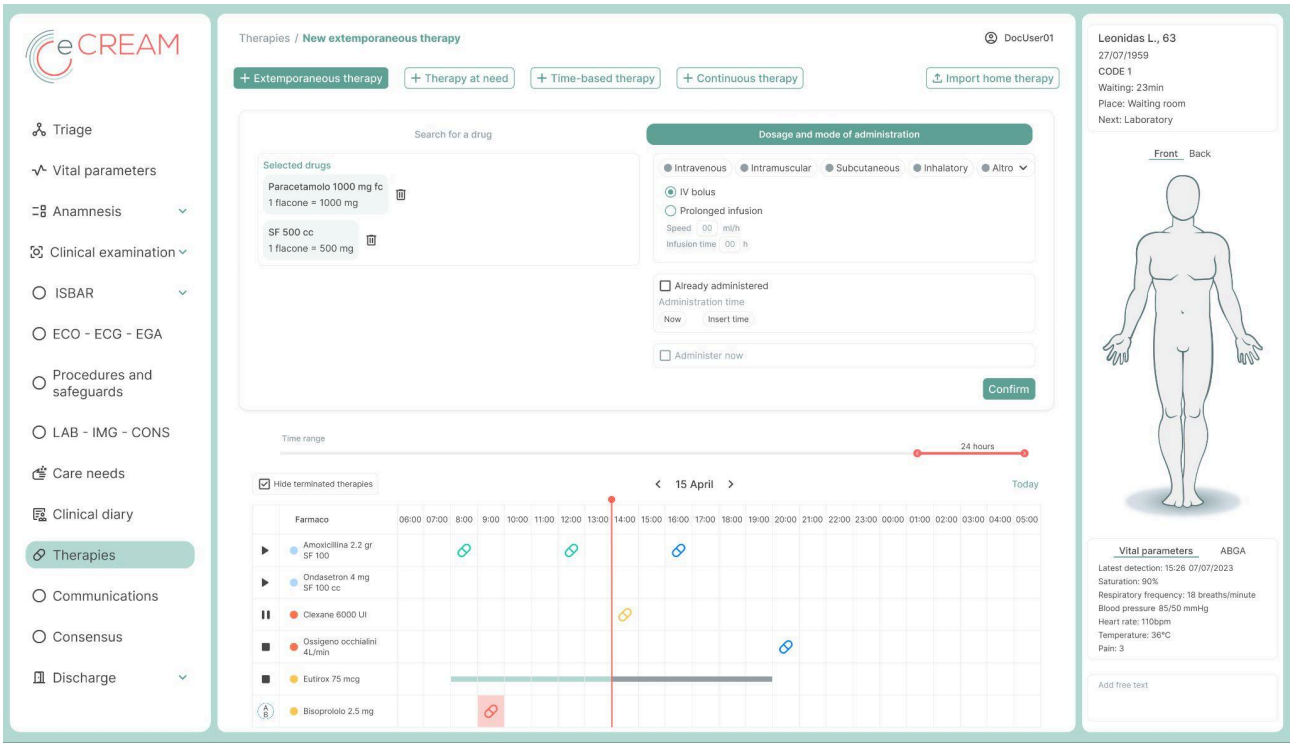


Figure 14. Screenshot of mock-up of the page Therapies.

Discharge

The page enables users to facilitate and document the discharge process for patients, recording essential details such as discharge instructions and prescribed medications. It includes subsections for Dispositions, Anamnesis, Clinical examination, Clinical diary, Procedures and safeguards, Care needs, and Preview and print. As shown in the figure below, the first subsection, Dispositions, provides a summary of the patient and hospital information, allergies and alerts noted on previous pages, and the discharge diagnosis from clinical staff. Each subsequent subsection offers a recap of information gathered from its respective page, presenting a comprehensive summary for easy review.

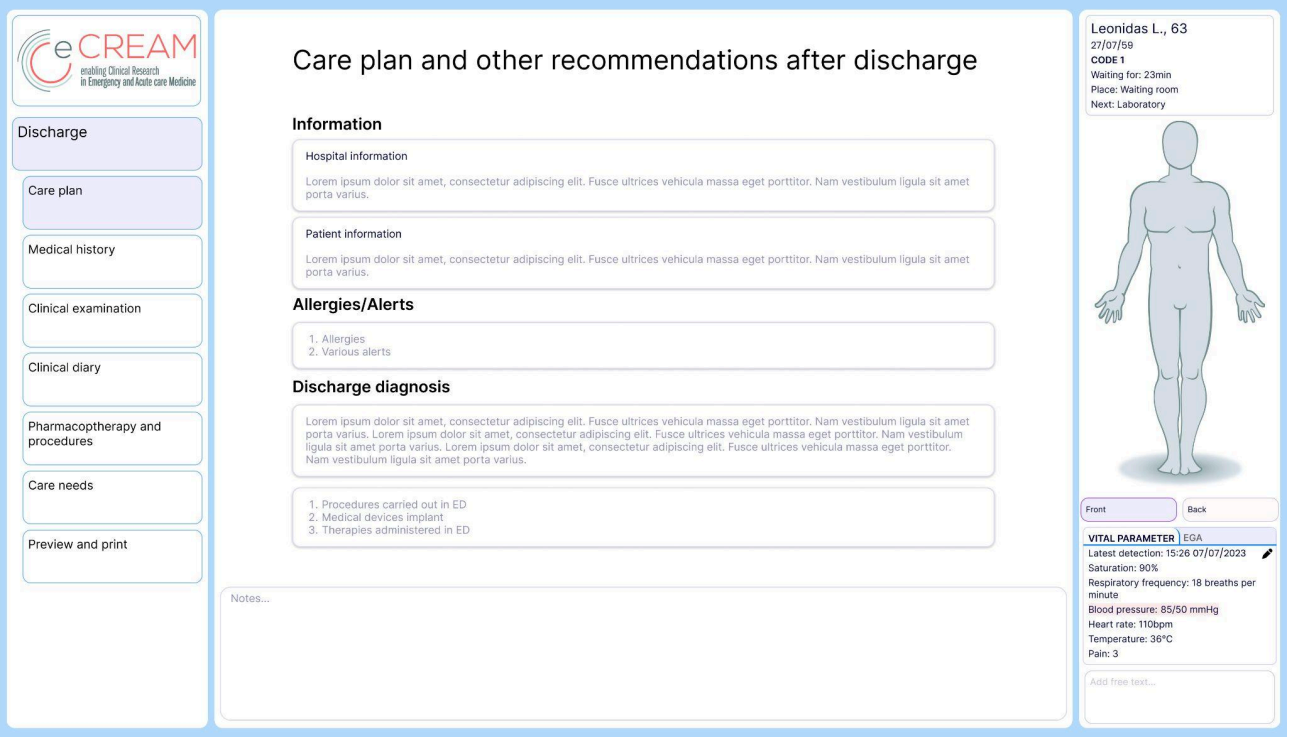


Figure 15. Screenshot of mock-up of the page Dispositions (Discharge).

Methodology

To align the platform with real-world needs and use contexts, we have been actively and continuously engaging clinicians and nurses in developing and assessing the prototype. The methodology in question is defined as user-centered design (UCD), an iterative design process that prioritizes users and their needs throughout every stage of the design process. This is done to foster continuous engagement with the user to better understand their preferences and requirements[4]. Each design iteration was reviewed collaboratively with users, allowing us to adjust and refine features based on their practical experience.

Paper Prototyping

In the last phase of the project, to enhance the design process for one of the pages of the platform, we decided to test an experimental methodology, paper prototyping. In this approach, designers and users create simple, paper-based versions of an interface[5]. We involved 10 clinicians and nurses from the Fenice Network[6] directly in the design of the clinical diary page, a component for documenting patient information and medical history.

[4] Interaction Design Foundation — IxDF. (2016, June 5). What is User Centered Design (UCD)?. Interaction Design Foundation — IxDF. https://www.interaction-design.org/literature/topics/user-centered-design
[5] Interaction Design Foundation — IxDF. (2017, June 30). What is Paper Prototyping?. Interaction Design Foundation — IxDF. https://www.interaction-design.org/literature/topics/paper-prototyping
[6] Fenice is the Italian Group for Clinical Research in Emergency Medicine, composed of several Italian Emergency Departments and coordinated by the Istituto di Ricerche Farmacologiche Mario Negri IRCCS https://fenicenetwork.marionegri.it/en/home-english/

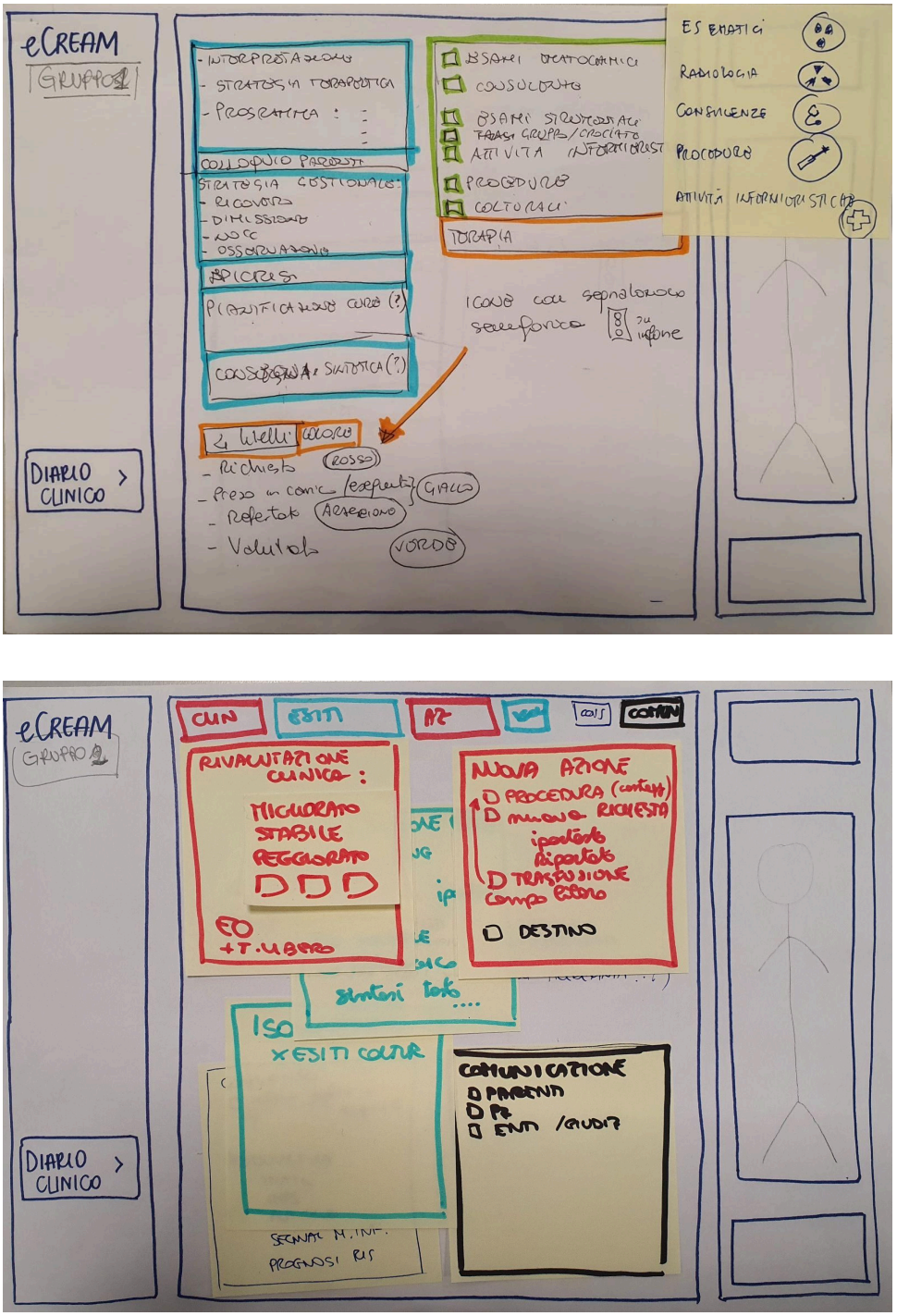


Figure 16. Results of the paper prototyping session.

By simply using pen and paper, we guided participants through the design process, allowing them to physically prototype and interact with the screens they created and provide immediate feedback. This methodology was highly effective in capturing user insights early in the design phase, as it encouraged participants to engage creatively and suggest changes without the constraints of a digital mock-up. The paper prototypes allowed us to quickly iterate on design ideas and gather valuable input on this page's content, layout, and functionalities with users.

Conclusion

Now that the preliminary research and planning have been completed, we are ready to begin developing the EHR platform. The groundwork has helped us to define the system's architecture, select a technology stack, and establish key principles to guide the development process.

It is worth remarking that the adoption of a hybrid architecture is aligned with the principles underlying the design of fine-grained systems like an EHR platform. It is a common good practice not to consider microservices as *the* default option, but as one of the possible architectural patterns to consider when designing a system architecture. In the deliverable, we presented the motivations that led us to adopt a hybrid architecture, combining the advantages of a monolith with those of microservices.

Henceforth, our objective is to implement our decisions progressively, with a particular focus on ease of use, security, and compliance with healthcare standards. As we dive into the development phase, it is important to recognize that, although the architectural framework established here provides a robust foundation, it remains adaptable.

According to the schedule of the WP3, the design process will continue along with the other activities of task 3.4. Incorporating user feedback through techniques such as paper prototyping has already demonstrated its worth, guaranteeing that the platform is user-centric and effectively addresses clinicians' requirements. The missing parts of the UI prototype will be discussed with the domain experts and developed.

We will persist in gathering requirements and refining our design based on continuous user insights and technological advancements. This dedication to adaptability allows us to respond to the evolving landscape of emergency and acute care medicine, ultimately enhancing clinical research and improving patient outcomes.